Project Report

Hospital Management System

1. Student Details

Name: Raj Singh

Roll No.DS23F2002385

Course: Modern Application Development – I

Program: IIT Madras BS Degree Program

About me: I am a student in the IIT Madras BS Degree program. I have a good interest in web application development and data-driven technologies. My work focuses on creating useful applications that combine learning, analytics, and user experience.

---

2. Project Details

Project Title: Hospital Management System

Problem Statement: Design and build a secure web-based system for managing patients, doctors, appointments, and treatment records. This will replace manual methods. The system must have different access levels for Admin, Doctor, and Patient

Approach: The application was built using **Flask** as the backend framework with a focus on **role-based access control (RBAC)**. The system handles secure authentication, programmatic database creation, appointment scheduling with conflict prevention, and complete patient history tracking.

---

3. AI/LLM Declaration

ChatGPT(GPT-5.1) were used only for clarity and organizing the structure of the code and only used about 20% of it. The final implementation logic, CRUD operations, validation, and integration were done manually. The frontend UI was created with Bootstrap, with only a little AI help in planning the content. The project is original and not copied from external sources.

4. Technologies and Frameworks Uses

Technology	Purpose
Flask	Backend routing & controllers
SQLite	Lightweight relational database
Jinja2	HTML template rendering
Bootstrap 5	Responsive frontend UI
Chart.js	Data visualization for admin dashboard
Flask-Login	Authentication & session management

---

5. System Architecture

Frontend

- Created using HTML, CSS, and Bootstrap
- Responsive on mobile
- Uses Jinja2 for dynamic pages
- Simple JS validations

Backend (Flask)

- Routing for all modules
- Login and logout system
- Role-based access control
- Appointment and treatment handling
- APIs for doctors and appointments

Database (SQLite)

Tables used:

- `user_accounts`
- `departments`
- `doctor_appointments`
- `patient_treatments`

---

6. DATABASE SCHEMA

UserAccounts

- user_id (PK)
- full_name
- email
- password
- role (admin/doctor/patient)
- dept_reference

Departments

- dept_id
- dept_name
- dept_description

DoctorAppointments

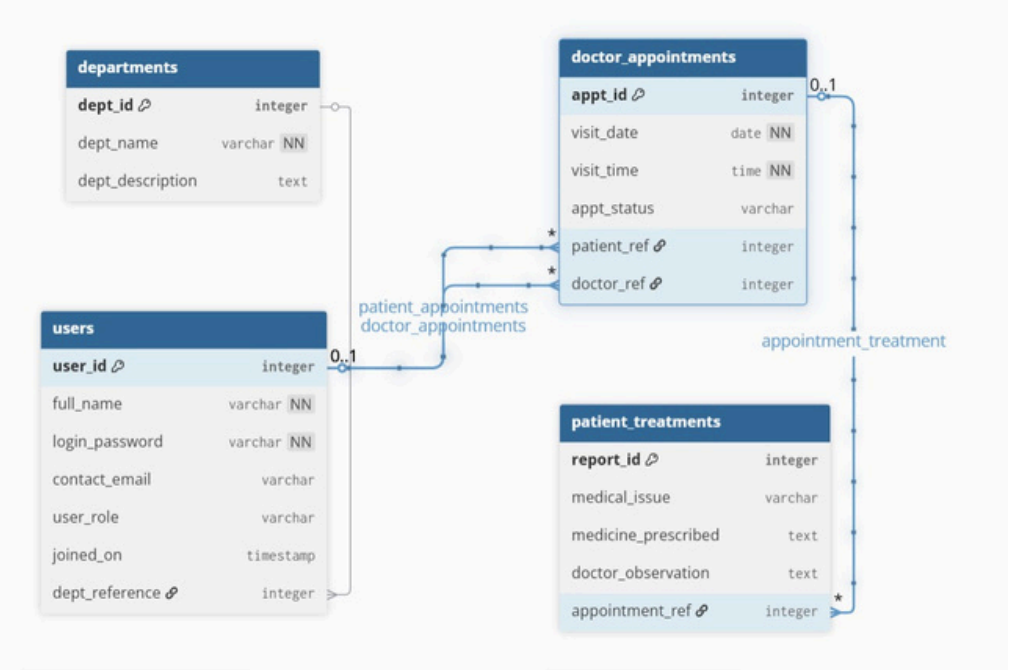
- appt_id
- visit_date
- visit_time
- appt_status
- patient_ref
- doctor_ref

PatientTreatment

- report_id
- medical_issue
- medicine_prescribed
- doctor_observation
- appointment_ref

ER Diagram Summary

- One department → many doctors
- One doctor → many appointments
- One appointment → one treatment
- One patient → many appointments



ER Diagram

7. CORE FEATURES IMPLEMENTED

Admin

- Add/edit doctor
- Add department
- View doctors/patients
- View appointments
- Check patient history
- Charts for basic stats

Patient

- Register/login
- Search doctor
- Book appointment
- Cancel/reschedule
- View previous visits

Doctor

- View assigned appointments
- Mark visits completed/cancelled
- Add diagnosis and medicines
- See patient history

---

8. API ENDPOINTS

GET /api/doctors–Fetch all doctors
POST /api/department – Add department
PUT /api/appointment/<id> – Update appointment
DELETE /api/appointment/<id> – Delete appointment

9.SECURITY & VALIDATION

Frontend

- Required fields
- Email check
- Password length

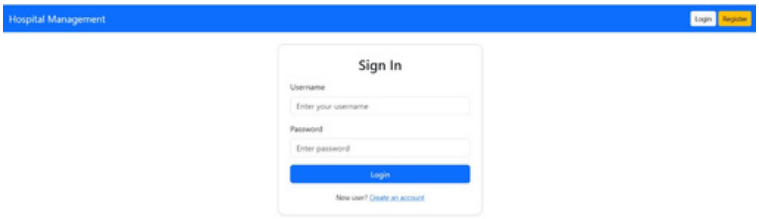
Backend

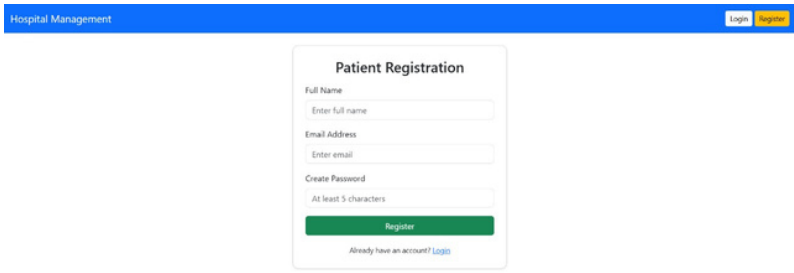
- Prevent duplicate emails
- Prevent double booking
- Protect routes using roles

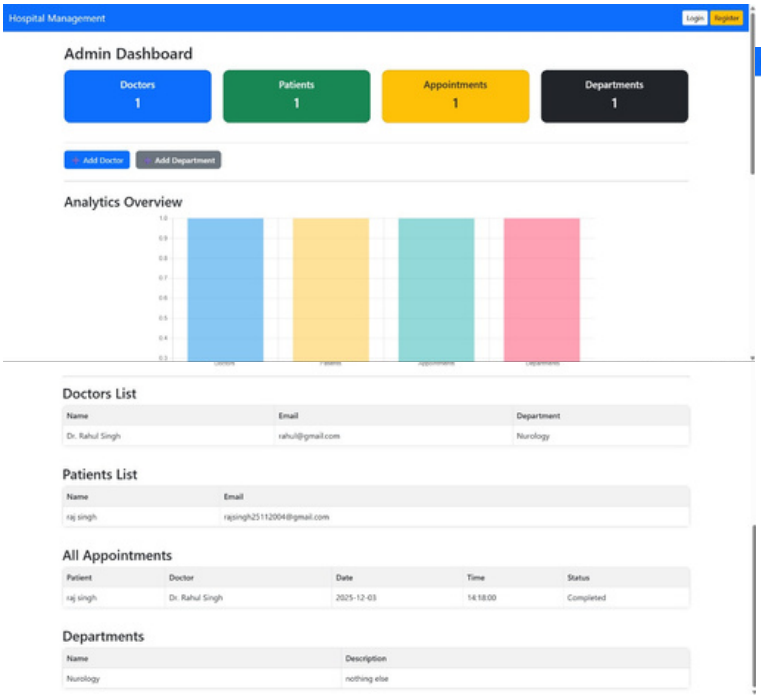
Password Security

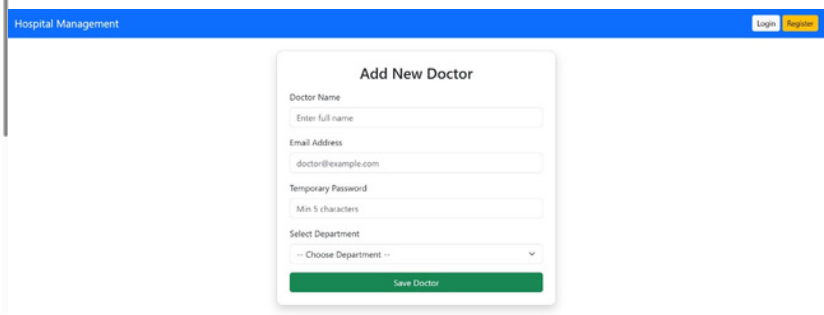
- Password hashing
- Flask-Login sessions

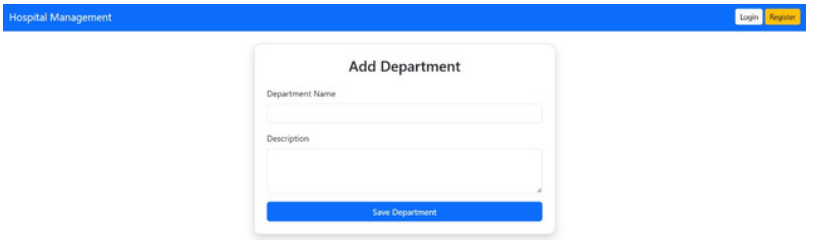
Screenshots of Web App











10. CONCLUSION

The HMS successfully manages login, doctor management, appointment booking, treatment storage, and basic charts. It cuts down on manual work and keeps all data organized.

The system can be improved in the future with notifications, cloud hosting, and better scheduling.

11. HMS Project Video Link

HMS Project Video Link